

2977. Minimum Cost to Convert String II

Solved 

Hard

 Topics

 Companies

 Hint

You are given two **0-indexed** strings `source` and `target`, both of length `n` and consisting of **lowercase** English characters. You are also given two **0-indexed** string arrays `original` and `changed`, and an integer array `cost`, where `cost[i]` represents the cost of converting the string `original[i]` to the string `changed[i]`.

You start with the string `source`. In one operation, you can pick a **substring** `x` from the string, and change it to `y` at a cost of `z` **if** there exists **any** index `j` such that `cost[j] == z`, `original[j] == x`, and `changed[j] == y`. You are allowed to do **any** number of operations, but any pair of operations must satisfy **either** of these two conditions:

- The substrings picked in the operations are `source[a..b]` and `source[c..d]` with either `b < c` **or** `d < a`. In other words, the indices picked in both operations are **disjoint**.
- The substrings picked in the operations are `source[a..b]` and `source[c..d]` with `a == c` **and** `b == d`. In other words, the indices picked in both operations are **identical**.

Return the **minimum** cost to convert the string `source` to the string `target` using **any** number of operations. If it is impossible to convert `source` to `target`, return `-1`.

Note that there may exist indices `i, j` such that `original[j] == original[i]` and `changed[j] == changed[i]`.

Example 1:

Input: source = "abcd", target = "acbe", original = ["a","b","c","c","e","d"],
changed = ["b","c","b","e","b","e"], cost = [2,5,5,1,2,20]

Output: 28

Explanation: To convert "abcd" to "acbe", do the following operations:

- Change substring source[1..1] from "b" to "c" at a cost of 5.
- Change substring source[2..2] from "c" to "e" at a cost of 1.
- Change substring source[2..2] from "e" to "b" at a cost of 2.
- Change substring source[3..3] from "d" to "e" at a cost of 20.

The total cost incurred is $5 + 1 + 2 + 20 = 28$.

It can be shown that this is the minimum possible cost.

Example 2:

Input: source = "abcdefgh", target = "acdeeghh", original =
["bcd","fgh","thh"], changed = ["cde","thh","ghh"], cost = [1,3,5]

Output: 9

Explanation: To convert "abcdefgh" to "acdeeghh", do the following operations:

- Change substring source[1..3] from "bcd" to "cde" at a cost of 1.
- Change substring source[5..7] from "fgh" to "thh" at a cost of 3. We can do this operation because indices [5,7] are disjoint with indices picked in the first operation.
- Change substring source[5..7] from "thh" to "ghh" at a cost of 5. We can do this operation because indices [5,7] are disjoint with indices picked in the first operation, and identical with indices picked in the second operation.

The total cost incurred is $1 + 3 + 5 = 9$.

It can be shown that this is the minimum possible cost.

Example 3:

Input: source = "abcdefgh", target = "addddddd", original = ["bcd","defgh"],
changed = ["ddd","dddd"], cost = [100,1578]

Output: -1

Explanation: It is impossible to convert "abcdefgh" to "addddddd".

If you select substring source[1..3] as the first operation to change "abcdefgh" to "adddefgh", you cannot select substring source[3..7] as the second operation because it has a common index, 3, with the first operation. If you select substring source[3..7] as the first operation to change "abcdefgh" to "abcdddd", you cannot select substring source[1..3] as the second operation because it has a common index, 3, with the first operation.

Constraints:

- `1 <= source.length == target.length <= 1000`
- `source`, `target` consist only of lowercase English characters.
- `1 <= cost.length == original.length == changed.length <= 100`
- `1 <= original[i].length == changed[i].length <= source.length`
- `original[i]`, `changed[i]` consist only of lowercase English characters.
- `original[i] != changed[i]`
- `1 <= cost[i] <= 106`

Python:

DP + Dijkstra. Runtime 100% (+ solutions with Trie and Floyd-Warshall)

class Solution:

def minimumCost(self, source: str, target: str, original: List[str], changed: List[str], cost: List[int]) -> int:

```
graph, dict_costs = defaultdict(dict), {}  
for u, v, c in zip(original, changed, cost):  
    graph[u][v] = min(c, graph[u].get(v, inf))
```

```
def calc_cost(source, target):  
    if source in dict_costs:  
        return dict_costs[source].get(target, inf)  
    heap, memo = [(0, source)], {source : 0}  
    while heap:  
        cost_current, word = heappop(heap)  
        if cost_current == memo[word]:  
            for neighbor, cost_neighbor in graph[word].items():  
                cost_new = cost_neighbor + cost_current
```

```

        if cost_new < memo.get(neighbor, inf):
            memo[neighbor] = cost_new
            heappush(heap, (cost_new, neighbor))
    dict_costs[source] = memo
    return memo.get(target, inf)

n = len(source)
set_len = sorted({*map(len, original)})
dp = [inf] * (n+1)
dp[0] = 0
for start in range(n):
    if dp[start] == inf:
        continue
    if source[start] == target[start]:
        dp[start + 1] = min(dp[start + 1], dp[start])
    for l in set_len:
        end = start + l
        if end > n:
            break
        s = source[start:end]
        if s in graph:
            dp[end] = min(dp[end], dp[start] + calc_cost(s, target[start:end]))
return -1 if dp[-1] == inf else dp[-1]

```

JavaScript:

```

/**
 * @param {string} source
 * @param {string} target
 * @param {string[]} original
 * @param {string[]} changed
 * @param {number[]} cost
 * @return {number}
 */
var minimumCost = function(source, target, original, changed, cost) {
    const index = {};
    original.forEach(o => {
        if (!(o in index)) {
            index[o] = Object.keys(index).length;
        }
    });
    changed.forEach(c => {
        if (!(c in index)) {
            index[c] = Object.keys(index).length;
        }
    });
}

```

```

});

const n = Object.keys(index).length;
const dis = new Array(n).fill(null).map(() => new
Array(n).fill(Number.POSITIVE_INFINITY));

for (let i = 0; i < cost.length; ++i) {
    dis[index[original[i]]][index[changed[i]]] =
Math.min(dis[index[original[i]]][index[changed[i]]], cost[i]);
}

for (let k = 0; k < n; ++k) {
    for (let i = 0; i < n; ++i) {
        if (dis[i][k] < Number.POSITIVE_INFINITY) {
            for (let j = 0; j < n; ++j) {
                if (dis[k][j] < Number.POSITIVE_INFINITY) {
                    dis[i][j] = Math.min(dis[i][j], dis[i][k] + dis[k][j]);
                }
            }
        }
    }
}

const substrLengths = new Set(original.map(o => o.length));

const dp = new Array(target.length + 1).fill(Number.POSITIVE_INFINITY);
dp[0] = 0;

for (let i = 0; i < target.length; ++i) {
    if (dp[i] === Number.POSITIVE_INFINITY) {
        continue;
    }

    if (target[i] === source[i]) {
        dp[i + 1] = Math.min(dp[i + 1], dp[i]);
    }

    for (const t of substrLengths) {
        if (i + t >= dp.length) {
            continue;
        }

        const subSource = source.substring(i, i + t);
        const subTarget = target.substring(i, i + t);

```

```

        const c1 = (subSource in index) ? index[subSource] : -1;
        const c2 = (subTarget in index) ? index[subTarget] : -1;

        if (c1 >= 0 && c2 >= 0 && dis[c1][c2] < Number.POSITIVE_INFINITY) {
            dp[i + t] = Math.min(dp[i + t], dp[i] + dis[c1][c2]);
        }
    }
}

return dp[dp.length - 1] !== Number.POSITIVE_INFINITY ? dp[dp.length - 1] : -1;
};

```

Java:

```

class Solution {
    // Standard Trie Node
    class TrieNode {
        TrieNode[] next = new TrieNode[26];
        int id = -1; // Unique ID for this string (-1 if not a complete word)
    }

    private int uniqueIDCounter = 0;

    public long minimumCost(String source, String target, String[] original, String[] changed, int[] cost) {
        // 1. Build Trie and Map Strings to Integer IDs
        TrieNode root = new TrieNode();
        for (String s : original) insert(root, s);
        for (String s : changed) insert(root, s);

        // 2. Initialize Distance Matrix (Graph)
        int numNodes = uniqueIDCounter;
        long[][] dist = new long[numNodes][numNodes];
        for (int i = 0; i < numNodes; i++) {
            Arrays.fill(dist[i], Long.MAX_VALUE);
            dist[i][i] = 0;
        }

        // Fill initial costs
        for (int i = 0; i < cost.length; i++) {
            int u = getID(root, original[i]);
            int v = getID(root, changed[i]);
            dist[u][v] = Math.min(dist[u][v], (long) cost[i]);
        }
    }
}

```

```
// 3. Floyd-Warshall (All-Pairs Shortest Path)
for (int k = 0; k < numNodes; k++) {
    for (int i = 0; i < numNodes; i++) {
        if (dist[i][k] == Long.MAX_VALUE) continue;
        for (int j = 0; j < numNodes; j++) {
            if (dist[k][j] == Long.MAX_VALUE) continue;
            dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
        }
    }
}
```

```
// 4. Dynamic Programming
int n = source.length();
long[] dp = new long[n + 1];
Arrays.fill(dp, Long.MAX_VALUE);
dp[0] = 0;
```

```
for (int i = 0; i < n; i++) {
    if (dp[i] == Long.MAX_VALUE) continue;
```

```
    // Option A: Character Match (Free)
    if (source.charAt(i) == target.charAt(i)) {
        dp[i + 1] = Math.min(dp[i + 1], dp[i]);
    }
```

```
    // Option B: Trie Search for Substring Replacement
    TrieNode p1 = root; // Pointer for source
    TrieNode p2 = root; // Pointer for target
```

```
    // Try matching substrings starting at 'i' of various lengths
    for (int j = i; j < n; j++) {
        int charS = source.charAt(j) - 'a';
        int charT = target.charAt(j) - 'a';
```

```
        p1 = p1.next[charS];
        p2 = p2.next[charT];
```

```
        // If either path falls off the Trie, we can't match further
        if (p1 == null || p2 == null) break;
```

```
        // If both pointers point to valid word IDs, check cost
        if (p1.id != -1 && p2.id != -1) {
            if (dist[p1.id][p2.id] != Long.MAX_VALUE) {
```

```

        dp[j + 1] = Math.min(dp[j + 1], dp[i] + dist[p1.id][p2.id]);
    }
}
}

return dp[n] == Long.MAX_VALUE ? -1 : dp[n];
}

private void insert(TrieNode root, String s) {
    TrieNode node = root;
    for (char c : s.toCharArray()) {
        if (node.next[c - 'a'] == null) {
            node.next[c - 'a'] = new TrieNode();
        }
        node = node.next[c - 'a'];
    }
    if (node.id == -1) {
        node.id = uniqueIDCounter++;
    }
}

private int getID(TrieNode root, String s) {
    TrieNode node = root;
    for (char c : s.toCharArray()) {
        node = node.next[c - 'a'];
    }
    return node.id;
}
}

```